LEHRSTUHL FÜR RECHNERARCHITEKTUR UND PARALLELE SYSTEME

Grundlagenpraktikum: Rechnerarchitektur

Tricorn-Fraktal (A212)
Projektaufgabe – Aufgabenbereich Bildverarbeitung

1 Organisatorisches

Auf den folgenden Seiten finden Sie die Aufgabenstellung zu Ihrer Projektaufgabe für das Praktikum. Die Rahmenbedingungen für die Bearbeitung werden in der Praktikumsordnung festgesetzt, die Sie über Artemis[1] aufrufen können.

Wie in der Praktikumsordnung beschrieben, sind die Aufgaben in bestimmten Punkten relativ offen gestellt. Besprechen Sie diese innerhalb Ihrer Gruppe und konkretisieren Sie die Aufgabenstellung. Die Teile der Aufgabe, in denen C-Code anzufertigen ist, sind in C nach dem C17-Standard zu schreiben.

Der **Abgabetermin** ist **Sonntag 19. Juli 2026, 23:59 Uhr**. Die Abgabe erfolgt per Artemis in das für Ihre Gruppe eingerichtete Projektrepository auf dem `main` Branch. Bitte beachten Sie die in der `README.md` angegebene Liste von abzugebenden Dateien.

Die **Abschlusspräsentationen** finden in der Zeit vom **17.08.2026 – 28.08.2026** statt. Weitere Informationen werden noch bekannt gegeben. Die Folien für die Präsentation sind am obigen Abgabetermin abzugeben; nachträgliche Änderungen sind nicht zulässig. Detaillierte formale Vorgaben zu den Folien finden Sie in der Praktikumsordnung.

Bei Fragen/Unklarheiten in Bezug auf den Ablauf und die Aufgabenstellung wenden Sie sich bitte an Ihre:n Tutor:in.

Wir wünschen Ihnen viel Erfolg und Freude bei der Bearbeitung Ihrer Aufgabe!

Mit freundlichen Grüßen
Die Praktikumsleitung

[1] `https://artemis.tum.de`

2 Tricorn-Fraktal

2.1 Überblick

Im Zuge Ihrer Projektaufgabe werden Sie theoretisches Wissen aus der Mathematik im Anwendungszusammenhang verwenden, um einen Algorithmus in C zu implementieren. Sie konzentrieren sich dabei auf eine Aufgabe im Bereich des *Image Processing*.

2.2 Funktionsweise

In dieser Aufgabe befassen Sie sich mit dem Multicorn-Fraktal. Diese ist gegeben durch iterative Anwendung der komplexen Gleichung

$$z_{i+1} = \overline{z_i}^m + c \; (i \geq 0)$$

und Betrachtung des Wertes z_i innerhalb der komplexen Ebene. Dabei wählt man $z_0 = 0$ und Real- sowie Imaginärteil von c gemäß der Koordinaten in der komplexen Ebene. Durch Variation von c und Betrachtung von z_n für große n lässt sich so das Fraktal in der komplexen Ebene darstellen. Im Rahmen der Aufgabe betrachten wir nur den Fall $m = 2$. Der Term $\overline{z_i}$ beschreibt die komplex konjugierte Zahl.

2.3 Aufgabenstellungen

Ihre Aufgaben lassen sich in die Bereiche Konzeption und Analyse (theoretisch) und Implementierung (praktisch) aufteilen. Antworten auf konzeptionelle Fragen sollten an den passenden Stellen in Ihrem Vortrag und Kurzbericht in angemessenem Umfang erscheinen. Die Antworten auf die Implementierungsaufgaben werden durch Ihren Code reflektiert.

2.3.1 Theoretischer Teil

- Berechnen Sie Real- sowie Imaginärteil von z_{i+1} in Abhängigkeit von Real- sowie Imaginärteil von z_i explizit für $m = 2$. Dies ist die Iterationsvorschrift, die Ihr Algorithmus verwenden soll.
- Sofern z_i beschränkt ist, färben Sie den Pixel, der zu Real- sowie Imaginärteil von c korrespondiert schwarz, da er zur Menge des Fraktals gehört. Wie verfahren Sie in dieser Hinsicht mit Pixeln, die zu Koordinaten korrespondieren, für die z_i divergiert?
- Der Algorithmus lässt sich leicht parallelisieren. Entwerfen Sie einen parallelen Algorithmus, der die SIMD-Einheit Ihrer Ziel-Architektur möglichst optimal verwendet.

2.3.2 Praktischer Teil

- Implementieren Sie in der Datei mit Ihrem C-Code die Funktion:

  ```
  void tricorn(float complex start, size_t width, ssize_t height, float
               res, unsigned n, bool color, unsigned char* img)
  ```

  Die Funktion berechnet das Tricorn-Fraktal ausgehend von `start` mit der gewünschten Auflösung (Schrittweite pro Bildpixel) `res` in der komplexen Ebene. Die Beträge von `width` und `height` bestimmen die Bilddimensionen, während ihre Vorzeichen die mathematische Schrittrichtung definieren. Als weitere Parameter erhält die Funktion die maximale Anzahl der Iterationen `n` sowie einen Zeiger auf einen Speicherbereich, der groß genug ist, um die berechneten Bitmap-Daten des Ergebnisses zu speichern. Allokieren Sie hierzu in Ihrem Rahmenprogramm einen Buffer passender Größe.

- Die farbliche Gestaltung wird über den Parameter `color` gesteuert. Falls dieser den Wert `false` hat, erfolgt die Berechnung des Pixelwerts gemäß der Vorschrift:

  $$\text{pixel} = \begin{cases} 0 & \text{falls } i = n \\ 255 - 4 \cdot (i \bmod 64) & \text{falls } i \neq n \end{cases}$$

  Nutzen Sie Ihre Vorarbeit aus den konzeptionellen Fragen, um zu verstehen, wofür der Wert i in dieser Vorschrift steht.
  Hat der Parameter `color` den Wert `true`, legen Sie Ihrer Ausgabe eine sinnvolle Farbpalette zugrunde.

- Verwenden Sie als Ausgabeformat die Windows Bitmap[2] (V3). Die Datei muss passend zum Vorzeichen von `height` entweder im `bottom-up`- oder im `top-down`-Format gespeichert werden.

2.3.3 Rahmenprogramm

Ihr Rahmenprogramm muss bei einem Aufruf die folgenden Optionen entgegennehmen und verarbeiten können. Wenn möglich soll das Programm sinnvolle Standardwerte definieren, sodass nicht immer alle Optionen gesetzt werden müssen. Wird eine Option mit Argument gesetzt, so ist das entsprechende Argument zu benutzen. Die Reihenfolge der Optionen bei einem gültigen Aufruf muss irrelevant sein. Wir empfehlen Ihnen, die Kommandozeilenparameter mittels `getopt_long`[3] zu parsen.

- `-V <Zahl>` — Die Implementierung, die verwendet werden soll. Hierbei soll mit `-V 0` Ihre Hauptimplementierung verwendet werden. Wenn diese Option nicht gesetzt wird, soll ebenfalls die Hauptimplementierung ausgeführt werden.

[2] `https://de.wikipedia.org/wiki/Windows_Bitmap`
[3] Siehe `man 3 getopt_long` für Details

- `-B <Zahl>` — Falls gesetzt, wird die Laufzeit der angegebenen Implementierung gemessen und ausgegeben. Das Argument dieser Option gibt die Anzahl an Wiederholungen des Funktionsaufrufs an, z. B. `-B 5`.
- `-s <Realteil>,<Imaginärteil>` — Startpunkt der Berechnung
- `-d <Zahl>,<Zahl>` — Breite und Höhe des Bildausschnitts
- `-n <Zahl>` — Maximale Anzahl an Iterationen pro Bildpixel
- `-r <Floating Point Zahl>` — Schrittweite pro Bildpixel
- `-o <Dateiname>` — Ausgabedatei
- `-C|--color` — Farbausgabe
- `-h|--help` — Eine Beschreibung aller Optionen des Programms und Verwendungsbeispiele werden ausgegeben und das Programm danach beendet. Die Beschreibung soll mit "Help Message" anfangen.
- `-t|--test` — Führt eine vordefinierte Testsuite aus, welche sämtliche von Ihnen verwendete Benchmarking-Szenarien gesammelt durchläuft. Das Programm gibt die jeweiligen Parameter, eine kurze Beschreibungen sowie die Ergebnisse in einer übersichtlichen und gut lesbaren Form aus und terminiert anschließend.

Sie dürfen weitere Optionen implementieren. Ihr Programm muss jedoch nur unter Verwendung der oben genannten Optionen verwendbar sein. Beachten Sie ebenfalls, dass Ihr Rahmenprogramm etwaige Randfälle korrekt abfangen muss und im Falle eines Fehlers mit einer Fehlermeldung, die auf den konkreten Fehler hinweist und einer kurzen Erläuterung zur Benutzung **terminieren** sollte. Fehlermeldungen jeglicher Art sollen auf `stderr` ausgegeben werden. Die Wertebereiche der Argumente dürfen nicht unnötig eingeschränkt werden.

2.4 Allgemeine Bewertungshinweise

Beachten Sie grundsätzlich alle in der Praktikumsordnung angegebenen Hinweise. Die folgende Liste konkretisiert einige der Bewertungspunkte:

- Diese PDF-Datei darf nicht verändert, gelöscht oder umbenannt werden.
- Die in `README.md` geforderte Struktur des Projektrepositorys muss eingehalten werden.
- Der Exit-Code Ihrer Implementierung muss die erfolgreiche Ausführung oder das Auftreten eines Fehlers widerspiegeln. Benutzen Sie dafür die Makros aus `stdlib.h`.
- Stellen Sie unbedingt sicher, dass Ihre Implementierung auf der Referenzplattform des Praktikums (`lxhalle`) kompiliert und vollständig funktionsfähig ist.

- Die Implementierung soll mit GCC/GNU as kompilieren. Verwenden Sie keinen Inline-Assembler. Achten Sie darauf, dass Ihr Programm keine x87-FPU- oder MMX-Instruktionen und SSE-Erweiterungen nur bis SSE4.2 verwendet. Andere ISA-Erweiterungen (z.B. AVX, BMI1) dürfen Sie nur benutzen, sofern Ihre Implementierung auch auf Prozessoren ohne derartige Erweiterungen lauffähig ist.
- Sie dürfen die angegebenen Funktionssignaturen, bis auf die Änderung des Funktionsnamens für die Benennung der unterschiedlichen Implementierungen (siehe nächster Punkt) *nicht* ändern.
- Verwenden Sie den angegebenen Funktionsnamen bzw. die angegebenen Funktionsnamen für Ihre Hauptimplementierung bzw. Hauptimplementierungen. Für alle weiteren Implementierungen gilt folgendes Schema: Hinter den Funktionsnamen wird das Suffix „`_V1`“, „`_V2`“, usw. angehängt.
- Denken Sie daran, das Laufzeitverhalten Ihres Codes zu testen (Sichere Programmierung, Korrektheit/Genauigkeit, Performanz) und behandeln Sie *alle möglichen Eingaben*, auch Randfälle. Ziehen Sie ggf. alternative Implementierungen als Vergleich heran.
- Eingabedateien, welche Sie generieren, um Ihre Implementierungen zu testen, sollten mit abgegeben werden; größere Eingaben sollten stattdessen stark komprimiert oder (bevorzugt) über ein abgegebenes Skript generierbar sein.
- Für die Folien der Abschlusspräsentation gelten die Vorgaben der Praktikumsordnung.